# Evaluation Metrics for Binary Cloud Segmentation

IoU · Dice · Precision · Recall

A standalone, framework-independent metrics module for the project **AI-Based Cloud Detection and Localization in Satellite Images**

| Item | Specification |
|---|---|
| Component | Evaluation and Metrics |
| Task type | Binary semantic segmentation (per-pixel: cloud vs clear) |
| Positive class | 1 = cloud pixel | 0 = clear / background pixel |
| Deliverable | `segmentation_metrics.py` — pure NumPy, no dataset, no training |
| Verification | `test_segmentation_metrics.py` — 12 hand-computed assertions, all passing |
| Integrates with | TensorFlow / Keras U-Net and PyTorch U-Net, without modification |

**Contents.** 1 Objective · 2 The pixel confusion matrix · 3 Metric definitions · 4 Input contract (y_true, y_pred) · 5 Probabilities to binary masks · 6 Edge cases and division by zero · 7 Aggregation: macro vs micro · 8 API reference · 9 Implementation · 10 Dummy example and verified output · 11 Integration with Keras and PyTorch · 12 Reporting guidance.

## 1 Objective and Deliverables

This module answers one question: *given a ground-truth cloud mask and a model's prediction for the same scene, how good is the prediction?* It is written to be finished and tested before the segmentation network exists, so that training work and evaluation work can proceed in parallel and the numbers reported at the end are never in doubt.

Three design rules follow from that:

- **No dataset, no model, no training.** The module imports only NumPy. It never loads an image and never touches a checkpoint.
- **Framework-independent.** Every entry point accepts NumPy arrays, Python lists, `torch.Tensor` or `tf.Tensor`. Conversion is done internally by duck-typing, so neither PyTorch nor TensorFlow needs to be installed for the module to import.
- **Deterministic and explicit.** No hidden smoothing constant is added to a reported metric, no `NaN` can ever escape, and every zero-denominator case has a documented, tested answer.

## 2 Foundation: the Pixel Confusion Matrix

All four metrics are ratios of the same four counts. In segmentation these counts are taken **per pixel**, not per image: a 256x256 tile contributes 65,536 independent decisions. With cloud as the positive class:

| Count | Meaning in this project | Cost of this error |
|---|---|---|
| **TP** — true positive | Cloud pixel correctly masked as cloud. | — |
| **FP** — false positive | Clear pixel wrongly masked as cloud. Typically bright ground: snow, sand, salt flats, rooftops. | Usable ground data is thrown away. |
| **FN** — false negative | Cloud pixel missed. Typically thin cirrus, haze, or the soft edge of a cloud. | Contaminated pixels leak into downstream analysis. |

| Count | Meaning in this project | Cost of this error |
|---|---|---|
| **TN** — true negative | Clear pixel correctly left unmasked. | — |

*Table 1. The four pixel-level outcomes. Localization quality is read from the spatial distribution of FP and FN, not from a single scalar.*

**Why accuracy is not the headline metric.** Satellite scenes are heavily imbalanced: TN usually dominates. A model that predicts *clear everywhere* on a 10%-cloudy scene scores 0.90 accuracy and is worthless. Section 10 shows exactly this in the demo output: accuracy 0.9183 against IoU 0.3989 on the same prediction. Accuracy and specificity are computed by the module as sanity checks only; IoU and Dice are the metrics of record.

# 3 The Four Metrics, Defined for Binary Segmentation

## 3.1 IoU — Intersection over Union (Jaccard index)

$$\text{IoU} = |\text{Pred} \cap \text{True}| \,/\, |\text{Pred} \cup \text{True}| = \text{TP} \,/\, (\text{TP} + \text{FP} + \text{FN})$$

The area where the predicted cloud region and the true cloud region overlap, divided by the area they jointly cover. TN is absent from the formula, so the empty sky cannot inflate the score. It is the strictest of the four: a false positive and a false negative both land in the same denominator, so a model cannot trade one for the other to look better. **This is the primary metric for the project.** Range 0 to 1; 1 means the predicted mask and the true mask are the same set of pixels.

## 3.2 Dice coefficient (Sorensen-Dice, identical to pixel-wise F1)

$$\text{Dice} = 2 \times |\text{Pred} \cap \text{True}| \,/\, (|\text{Pred}| + |\text{True}|) = 2\text{TP} \,/\, (2\text{TP} + \text{FP} + \text{FN})$$

The intersection is counted twice, once against each mask, so Dice weights agreement more heavily than IoU does. It is always greater than or equal to IoU, and the two are locked together by an exact identity that the test suite verifies:

$$\text{Dice} = 2 \times \text{IoU} \,/\, (1 + \text{IoU})$$

Because the relationship is monotonic, Dice never ranks two models differently from IoU. It is reported because it is the convention in the segmentation literature, it is more forgiving on small or thin cloud fragments, and its differentiable form (soft Dice) is the standard U-Net training loss. Dice is mathematically the harmonic mean of precision and recall, which is why `f1_score` is provided as a plain alias.

## 3.3 Precision (positive predictive value)

$$\text{Precision} = \text{TP} \,/\, (\text{TP} + \text{FP})$$

Purity of the predicted mask: *of every pixel the model called cloud, what fraction really was cloud?* Low precision means the model is over-masking, flagging bright ground as cloud and discarding good data. Precision alone is trivially gamed — predict a single confident cloud pixel and precision is 1.0 — so it is only meaningful when read next to recall.

## 3.4 Recall (sensitivity, true positive rate)

$$\text{Recall} = \text{TP} \,/\, (\text{TP} + \text{FN})$$

Coverage of the true mask: *of every pixel that really was cloud, what fraction did the model find?* Low recall means thin cirrus and cloud boundaries are leaking through. Recall alone is gamed in the opposite direction — mask the whole image and recall is 1.0. The precision/recall pair therefore describes the **direction** of the error, while IoU and Dice describe its **magnitude**.

| Metric | Formula | Uses TN? | Answers | Read it when |
|---|---|---|---|---|
| IoU | `TP / (TP+FP+FN)` | No | Overall overlap quality (strict) | Reporting the headline result |
| Dice / F1 | `2TP / (2TP+FP+FN)` | No | Overall overlap quality (lenient) | Comparing against published work |
| Precision | `TP / (TP+FP)` | No | Is the mask over-extended? | FP rate looks high |
| Recall | `TP / (TP+FN)` | No | Is the mask under-extended? | Thin cloud is being missed |

| Metric | Formula | Uses TN? | Answers | Read it when |
|---|---|---|---|---|
| Accuracy | `(TP+TN) / all` | Yes | Fraction of pixels correct | Sanity check only — inflated by imbalance |
| Specificity | `TN / (TN+FP)` | Yes | Is clear sky preserved? | Quantifying data lost to over-masking |

*Table 2. Metric selection guide. The four required metrics ignore TN entirely, which is precisely why they survive class imbalance.*

# 4 Input Contract: y_true and y_pred

Every public function takes the ground truth first and the prediction second: `metric(y_true, y_pred)`. The two arguments differ in what they are allowed to contain.

| | y_true (ground truth) | y_pred (model output) |
|---|---|---|
| Content | Binary labels only: 0 or 1. Any other value raises `ValueError`. | Continuous probabilities in [0, 1], or raw logits with `from_logits=True`. |
| Dtype | Any integer or float dtype; cast internally to float64. | Any float dtype; cast internally to float64. |
| Thresholded? | No — it is already a decision. | Yes — see Section 5. |
| Shape | Must match y_pred after the channel axis is squeezed. | Must match y_true after the channel axis is squeezed. |

*Table 3. What each argument must contain.*

## 4.1 Accepted shapes

A private helper canonicalizes the input so the same call works whichever framework produced the array. A singleton axis is squeezed **only** for 4-D input, so a batch of one image is never silently collapsed into a single mask.

| Input shape | Interpretation | Canonical form | Produced by |
|---|---|---|---|
| `(H, W)` | One mask | `(1, H, W)` | A single tile, NumPy |
| `(N, H, W)` | Batch of N masks | `(N, H, W)` | Squeezed output |
| `(N, H, W, 1)` | Batch, channels-last | `(N, H, W)` | TensorFlow / Keras |
| `(N, 1, H, W)` | Batch, channels-first | `(N, H, W)` | PyTorch |
| `(N, H, W, C>1)` | Rejected | `ValueError` | Multi-class output — out of scope |

*Table 4. Shape handling. The test suite asserts that all four accepted layouts return an identical IoU for the same underlying data.*

**Requirement:** y_true and y_pred must describe the same pixel grid. If a shape mismatch survives canonicalization the module raises immediately rather than broadcasting — silent NumPy broadcasting is the single most common source of a wrong metric.

# 5 From Probabilities to Binary Masks

A U-Net with a 1-channel sigmoid head emits a continuous confidence map, one value per pixel. IoU, Dice, precision and recall are set operations and are undefined on continuous values, so the map must be hardened into a {0, 1} mask first. The rule is a single comparison:

$$\texttt{mask[i, j] = 1 if probability[i, j] >= threshold, else 0}$$

```
from segmentation_metrics import binarize

mask = binarize(probs,  threshold=0.5)                    # sigmoid output in [0, 1]
mask = binarize(logits, threshold=0.5, from_logits=True)  # raw logits -> sigmoid first
```

- **Sigmoid head (usual case).** Output is already in [0, 1]. Pass it straight in. Values outside [0, 1] raise a `ValueError` rather than being silently clipped — that error almost always means a missing activation.
- **Logits head.** Set `from_logits=True`; a numerically stable sigmoid is applied first. Thresholding a logit at 0.5 instead of 0.0 is a real and easy mistake, and this flag removes it.

- **Softmax 2-channel head.** Take channel 1 (the cloud channel) and treat it as the probability map: `probs[..., 1]`. Argmax over two channels is exactly equivalent to thresholding that channel at 0.5.
- **Boundary convention.** The comparison is >=, so a pixel at exactly 0.5 is classified as cloud. This is stated because it changes results on logit-0 pixels, as the test suite demonstrates.

### 5.1 Choosing the threshold

0.5 is the neutral default, not the optimal one. Raising the threshold increases precision and decreases recall; lowering it does the reverse. Sweep it once on the **validation** split, freeze the winner, and reuse that frozen value on the test split — tuning the threshold on test data inflates the reported score and invalidates the comparison.

```
from segmentation_metrics import threshold_sweep

best, table = threshold_sweep(val_masks, val_probs, optimize="iou")
# best  -> 0.6
# table -> {0.05: {...}, 0.10: {...}, ...} one metric dict per candidate threshold
```

## 6 Edge Cases and Division by Zero

Clear-sky tiles are common in satellite data, and on such a tile TP, FP and FN can all be zero. A naive implementation returns `0/0 = nan`, one `nan` poisons the batch mean, and the whole evaluation collapses. The module resolves every zero denominator explicitly.

| Situation | Counts | IoU | Dice | Precision | Recall | Rationale |
|---|---|---|---|---|---|---|
| Clear scene, nothing predicted | TP=FP=FN=0 | 1.0 | 1.0 | 1.0 | 1.0 | Perfect agreement. Configurable via `empty_score`. |
| Clear scene, cloud predicted | TP=0, FP>0, FN=0 | 0.0 | 0.0 | 0.0 | 1.0 | Every predicted pixel is wrong. |
| Cloudy scene, nothing predicted | TP=0, FP=0, FN>0 | 0.0 | 0.0 | 0.0 | 0.0 | Every cloud pixel was missed. |
| Perfect prediction | FP=FN=0, TP>0 | 1.0 | 1.0 | 1.0 | 1.0 | Upper bound. |
| Inverted prediction | TP=0 | 0.0 | 0.0 | 0.0 | 0.0 | Lower bound. |

*Table 5. Every zero-denominator outcome is defined and asserted in the test suite. No call can return NaN.*

The rule is implemented once, in a single helper, so all four metrics share it. A zero denominator carries exactly two possible meanings, and they are distinguished rather than collapsed:

```
def _safe_ratio(numerator, denominator, both_empty, empty_score):
    out = np.divide(numerator, denominator,
                    out=np.zeros_like(denominator, dtype=np.float64),
                    where=denominator > 0)
    return np.where(denominator > 0, out,
                    np.where(both_empty, empty_score, 0.0))
```

- `both_empty` is true (TP+FP+FN == 0): the scene has no cloud and none was predicted. The answer is correct, so `empty_score` (default 1.0) is returned. Set it to 0.0 for the stricter convention used by some benchmarks, or drop clear tiles from the evaluation set entirely and say so in the report.
- `both_empty` is false: one side is empty and the other is not, so the prediction is wholly wrong for that image and 0.0 is returned.

**Smoothing belongs in the loss, not in the metric.** Adding +1 to numerator and denominator is standard practice for a *differentiable training loss*, where it stabilises gradients. In a *reported metric* the same trick silently drags every score toward 1.0 and makes a bad model look acceptable. The smoothing constant in this module therefore appears only in `soft_dice_loss_torch`, never in the reporting functions.

## 7 Aggregation Across a Batch: Macro vs Micro

With more than one image, there are two defensible ways to reach a single number, and they disagree. The `reduction` argument selects between them.

| reduction | Procedure | Effect | Use for |
|---|---|---|---|
| "mean" (default, macro) | Compute the metric per image, then average the per-image values. | Every tile counts equally. A small missed cloud on an almost-clear tile hurts as much as a large error on a cloudy tile. | The headline number in the report. |
| "micro" | Pool TP/FP/FN over the whole set, then compute the metric once. | Every pixel counts equally, so large cloud fields dominate the score. | A dataset-wide pixel-level view. |
| "none" | Return one value per image. | No aggregation. | Error analysis: rank the worst tiles and inspect them. |

*Table 6. Aggregation modes. Whichever is chosen must be stated alongside the number — "IoU 0.72" is not reproducible without it.*

The gap between the two is diagnostic rather than cosmetic. In the demo of Section 10 the macro IoU is **0.3989** while the micro IoU is **0.5246** on identical predictions. The cause is the third tile: it is entirely clear, the model produced a scatter of false positives on it, and its per-image IoU is 0.0. Macro averaging gives that tile a full one-third vote; micro pooling buries its handful of wrong pixels among the thousands of correct pixels from the other two tiles. **Recommendation for this project: report macro as the headline, report micro alongside it, and investigate whenever the two diverge by more than a few points.**

# 8 API Reference

| Function | Returns | Purpose |
|---|---|---|
| `iou_score(y_true, y_pred, ...)` | float or array | Jaccard index for the cloud class. |
| `dice_coefficient(y_true, y_pred, ...)` | float or array | Dice / F1 coefficient. |
| `precision_score(y_true, y_pred, ...)` | float or array | TP / (TP + FP). |
| `recall_score(y_true, y_pred, ...)` | float or array | TP / (TP + FN). |
| `f1_score(...)` | float or array | Alias of dice_coefficient. |
| `accuracy_score(y_true, y_pred, ...)` | float | Sanity check only. |
| `specificity_score(y_true, y_pred, ...)` | float | TN / (TN + FP). |
| `compute_all_metrics(y_true, y_pred, ...)` | dict | Full suite plus TP/FP/FN/TN in one pass. Log this straight to CSV or TensorBoard. |
| `confusion_counts(y_true, y_pred, ...)` | dict | Raw TP/FP/FN/TN, globally or per image. |
| `binarize(y_pred, threshold, from_logits)` | ndarray | Probabilities or logits -> {0,1}. |
| `sigmoid(z)` | ndarray | Numerically stable logistic sigmoid. |
| `MetricsAccumulator(...)` | object | Streaming `.update()` / `.result()` over mini-batches; keeps macro and micro at once. |
| `threshold_sweep(y_true, y_pred, ...)` | (float, dict) | Best threshold plus the full sweep table. |
| `soft_dice_loss_torch(logits, targets)` | tensor | Differentiable training loss (PyTorch, lazily imported). |

*Table 7. Public surface of* `segmentation_metrics`*. Shared keyword arguments:* `threshold=0.5`*,* `from_logits=False`*,* `reduction='mean'`*,* `empty_score=1.0`*.*

# 9 Implementation

The computational core is reproduced below. The complete 675-line module, with full docstrings, input validation, the streaming accumulator and the threshold sweep, ships as `segmentation_metrics.py` alongside this report.

### 9.1 Counting

```
def confusion_counts(y_true, y_pred, threshold=0.5, from_logits=False, per_image=False):
    """Pixel-level TP / FP / FN / TN for the positive (cloud) class."""
    t, p = _prepare(y_true, y_pred, threshold, from_logits)   # validate + binarize
    axis = tuple(range(1, t.ndim))            # every axis except the batch axis

    tp = np.sum((t == 1) & (p == 1), axis=axis, dtype=np.float64)
    fp = np.sum((t == 0) & (p == 1), axis=axis, dtype=np.float64)
    fn = np.sum((t == 1) & (p == 0), axis=axis, dtype=np.float64)
    tn = np.sum((t == 0) & (p == 0), axis=axis, dtype=np.float64)

    if not per_image:
        tp, fp, fn, tn = tp.sum(), fp.sum(), fn.sum(), tn.sum()
    return {"tp": tp, "fp": fp, "fn": fn, "tn": tn}
```

### 9.2 The shared metric engine

```
def _metric(y_true, y_pred, kind, threshold, from_logits, reduction, empty_score):
    micro  = reduction == "micro"
    counts = confusion_counts(y_true, y_pred, threshold=threshold,
                              from_logits=from_logits, per_image=not micro)
    tp, fp, fn = (np.atleast_1d(counts[k]) for k in ("tp", "fp", "fn"))
    both_empty = (tp + fp + fn) == 0

    ratio = lambda num, den: _safe_ratio(num, den, both_empty, empty_score)

    if   kind == "iou":       values = ratio(tp,       tp + fp + fn)
    elif kind == "dice":      values = ratio(2.0 * tp, 2.0 * tp + fp + fn)
    elif kind == "precision": values = ratio(tp,       tp + fp)
    elif kind == "recall":    values = ratio(tp,       tp + fn)

    if micro:
        return float(values[0])
    return _reduce(values, "mean" if reduction == "mean" else reduction)
```

Writing the four metrics through one engine means the binarization rule, the shape rules and the zero-denominator policy cannot drift apart between them. Each public function is then a thin, documented wrapper:

```
def iou_score(y_true, y_pred, threshold=0.5, from_logits=False,
              reduction="mean", empty_score=1.0):
    """IoU = TP / (TP + FP + FN) for the cloud class."""
    return _metric(y_true, y_pred, "iou", threshold, from_logits, reduction, empty_score)
```

### 9.3 One-pass suite

```
def compute_all_metrics(y_true, y_pred, threshold=0.5, from_logits=False,
                        reduction="mean", empty_score=1.0, include_counts=True):
    common = dict(threshold=threshold, from_logits=from_logits)
    macro  = dict(reduction=reduction, empty_score=empty_score, **common)
    results = {
        "iou":         iou_score(y_true, y_pred, **macro),
        "dice":        dice_coefficient(y_true, y_pred, **macro),
        "precision":   precision_score(y_true, y_pred, **macro),
        "recall":      recall_score(y_true, y_pred, **macro),
        "accuracy":    accuracy_score(y_true, y_pred, **common),
        "specificity": specificity_score(y_true, y_pred, **common),
    }
    if include_counts:
        results.update({k: float(v) for k, v in
                        confusion_counts(y_true, y_pred, **common).items()})
    return results
```

## 10 Dummy Example and Verified Output

The module runs standalone: `python segmentation_metrics.py`. No dataset and no model are involved; the "predictions" are synthetic arrays.

### 10.1 A 4x4 mask that can be checked by hand

```
y_true = [[0, 0, 0, 0],          y_prob = [[0.01, 0.02, 0.03, 0.04],
          [0, 1, 1, 0],                    [0.10, 0.90, 0.95, 0.80],   <- FP at (1,3)
          [0, 1, 1, 0],                    [0.05, 0.85, 0.40, 0.02],   <- FN at (2,2)
          [0, 0, 0, 0]]                    [0.00, 0.01, 0.02, 0.03]]

counts  -> TP=3  FP=1  FN=1  TN=11
  iou         = 0.6000        dice        = 0.7500
  precision   = 0.7500        recall      = 0.7500
  accuracy    = 0.8750        specificity = 0.9167
```

| Metric | Arithmetic | Expected | Module output |
|---|---|---|---|
| IoU | 3 / (3 + 1 + 1) = 3/5 | 0.6000 | 0.6000 |
| Dice | (2×3) / (2×3 + 1 + 1) = 6/8 | 0.7500 | 0.7500 |
| Precision | 3 / (3 + 1) = 3/4 | 0.7500 | 0.7500 |
| Recall | 3 / (3 + 1) = 3/4 | 0.7500 | 0.7500 |
| Accuracy | (3 + 11) / 16 = 14/16 | 0.8750 | 0.8750 |
| Specificity | 11 / (11 + 1) = 11/12 | 0.9167 | 0.9167 |
| Identity check | 2×0.6 / (1 + 0.6) = 1.2/1.6 | 0.7500 (= Dice) | 0.7500 |

*Table 8. Hand arithmetic against module output. Every row matches, including the Dice = 2·IoU/(1+IoU) identity.*

## 10.2 A batch of three tiles, channels-last (3, 32, 32, 1)

Tile 1 holds a compact cloud block, tile 2 a thin cloud band, and tile 3 is completely clear. Gaussian noise is added to the synthetic confidence map so the prediction is imperfect and the aggregation modes separate.

```
shapes: y_true (3, 32, 32, 1), y_pred (3, 32, 32, 1)
per-image IoU       : [0.720  0.477  0.000]      <- tile 3 is clear but got false positives
macro IoU (mean)    : 0.3989
micro IoU (pooled)  : 0.5246
  iou         = 0.3989        dice        = 0.4943
  precision   = 0.4262        recall      = 0.6010
  accuracy    = 0.9183        specificity = 0.9194
  tp = 277    fp = 223    fn = 28     tn = 2544
```

Three things are readable straight off this output, and all three are the point of the module. **Accuracy 0.9183 against IoU 0.3989** on the same prediction is the imbalance trap of Section 2 made concrete. **Recall 0.6010 well above precision 0.4262** says the model is over-masking: it finds most of the cloud but paints far beyond its edges, which points at raising the threshold. **Macro 0.3989 against micro 0.5246** is the clear tile of Section 7 exercising its full one-third vote.

## 10.3 Edge cases, accumulator and threshold sweep

```
clear scene, nothing predicted    -> IoU    = 1.0   (perfect agreement)
clear scene, everything predicted -> IoU    = 0.0   (all false positives)
cloudy scene, nothing predicted   -> recall = 0.0   (all false negatives)
no NaN is ever produced.

macro IoU 0.3989 | micro IoU 0.5246 | images 3      (streaming, 1 tile per update)

  t=0.30  IoU=0.2557  Dice=0.3606  P=0.2567  R=0.6595
  t=0.40  IoU=0.3349  Dice=0.4383  P=0.3412  R=0.6465
  t=0.50  IoU=0.3989  Dice=0.4943  P=0.4262  R=0.6010
  t=0.60  IoU=0.4516  Dice=0.5373  P=0.5263  R=0.5532
  t=0.70  IoU=0.4275  Dice=0.5210  P=0.5796  R=0.4767
best threshold by IoU: 0.60
```

The sweep shows the precision/recall trade-off behaving exactly as Section 5.1 predicts: precision climbs from 0.2567 to 0.5796 as the threshold rises, recall falls from 0.6595 to 0.4767, and IoU peaks in between at 0.60. The accumulator fed one tile at a time reproduces the single-pass macro and micro figures to nine decimal places.

## 10.4 Verification suite

`test_segmentation_metrics.py` contains twelve tests whose expected values are derived by hand from the confusion counts, so they prove the formulas rather than freezing whatever the code currently prints.

```
$ python test_segmentation_metrics.py
  PASS  test_accumulator_matches_single_pass     PASS  test_logits_path_matches_manual_sigmoid
  PASS  test_confusion_counts_hand_checked       PASS  test_macro_vs_micro_differ_as_expected
  PASS  test_dice_iou_identity                   PASS  test_perfect_and_inverted_prediction
  PASS  test_empty_mask_edge_cases_no_nan        PASS  test_shape_variants_agree
  PASS  test_f1_equals_dice                      PASS  test_threshold_changes_the_decision
  PASS  test_input_validation_rejects_bad_data   PASS  test_threshold_sweep_picks_the_maximum

12/12 tests passed.
```

# 11 Connecting the Module to a U-Net

Nothing in the module changes when the trained model arrives. Only the source of `y_pred` changes.

## 11.1 TensorFlow / Keras

```
import numpy as np
from segmentation_metrics import compute_all_metrics, MetricsAccumulator

# model output: (N, H, W, 1) sigmoid probabilities -- accepted as-is
probs = model.predict(val_images)
print(compute_all_metrics(val_masks, probs, threshold=0.5))

# Or stream over a tf.data pipeline to keep memory flat:
acc = MetricsAccumulator(threshold=0.5)
for images, masks in val_dataset:
    acc.update(masks, model(images, training=False))    # tf.Tensor converted internally
print(acc.result())

# Per-epoch logging without writing a custom Keras metric:
class MetricsCallback(tf.keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs=None):
        acc = MetricsAccumulator()
        for images, masks in val_dataset:
            acc.update(masks, self.model(images, training=False))
        r = acc.result()
        print(f"epoch {epoch}: IoU {r['macro_iou']:.4f}  Dice {r['macro_dice']:.4f}")
```

## 11.2 PyTorch

```
import torch
from segmentation_metrics import MetricsAccumulator, soft_dice_loss_torch

# --- training: differentiable loss, no thresholding, smoothing allowed ---
loss = soft_dice_loss_torch(logits, masks)          # combine with BCEWithLogitsLoss
loss.backward()

# --- evaluation: hard masks, no smoothing ---
model.eval()
acc = MetricsAccumulator(threshold=0.5, from_logits=True)   # model emits raw logits
with torch.no_grad():
    for images, masks in val_loader:                # masks: (N, 1, H, W)
        acc.update(masks, model(images))            # torch.Tensor converted internally
print(acc.result())      # macro_iou, micro_iou, macro_dice, precision, recall, tp/fp/fn/tn
```

**Keep the two paths separate.** The training loss is a smoothed, differentiable approximation of Dice computed on probabilities. The reported metric is an exact, unsmoothed set operation computed on thresholded masks. They will not agree numerically, and they are not supposed to.

# 12 Reporting Guidance for the Final Write-up

- **Report all four metrics together, never one alone.** IoU and Dice give the magnitude of the error; precision and recall give its direction. A single number cannot distinguish over-masking from under-masking.

- **State the protocol next to every number.** Threshold, reduction mode (macro or micro), the `empty_score` convention, and whether clear-only tiles were included. "IoU 0.72" is not reproducible; "macro IoU 0.72 at threshold 0.6, empty_score 1.0, clear tiles included" is.
- **Freeze the threshold on validation data.** Sweep once on the validation split, fix the winner, and use it unchanged on the test split.
- **Publish the confusion counts.** TP, FP, FN and TN let a reader recompute every metric and catch a mistake that a rounded score would hide.
- **Include a per-image distribution, not only the mean.** Run with `reduction="none"` and show the histogram plus the worst tiles. A mean of 0.70 from a tight cluster and a mean of 0.70 from a bimodal split are different models with different failure modes.
- **Treat accuracy as a footnote.** Include it if the rubric asks, but never lead with it — Section 10.2 shows 0.9183 accuracy sitting on top of a 0.3989 IoU.

---

*Deliverables:* `segmentation_metrics.py` *(module, 675 lines)* · `test_segmentation_metrics.py` *(12 passing tests) · this report. Dependencies: NumPy only. No dataset was used and no model was trained.*